

Project Executable Files

Date	2 July 2026
Team ID	
Project Name	EduGenie
Maximum Marks	3 Marks

Project Executable Files

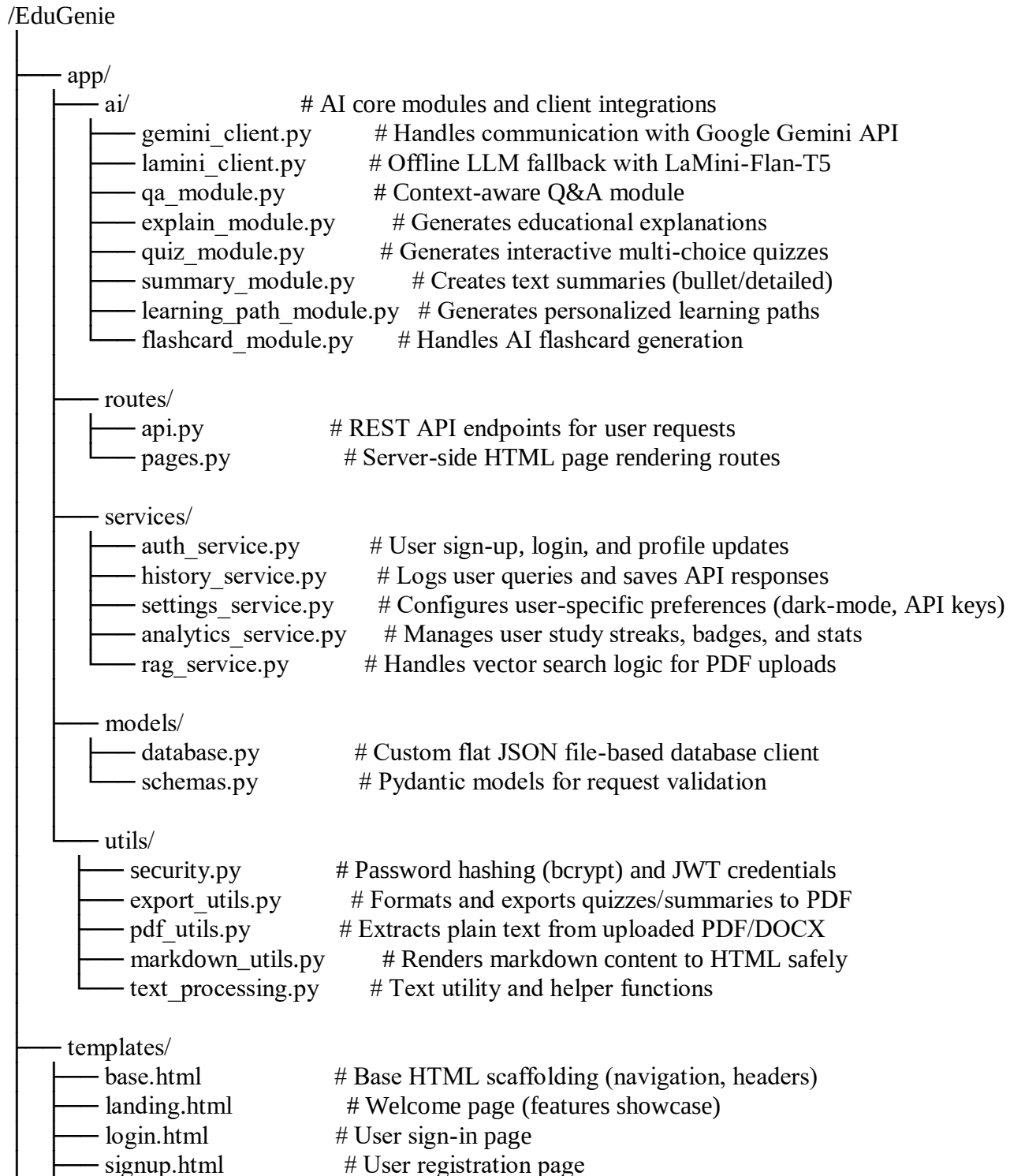
This section presents the executable and deployable files for the EduGenie platform. It includes the source code, configuration files, dependency specifications, and other supporting resources required to run and evaluate the application. The files are organized to facilitate easy deployment, verification, and assessment of the project's functionality.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	NA
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	Yes
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

The following section presents the file and folder structure of the EduGenie project. It shows how the application's source code, AI modules, database, templates, static assets, configuration files, and documentation are organized to ensure a modular, maintainable, and scalable architecture.



— dashboard/	# Core educational pages
— base_dashboard.html	# Sidebar layout template for dashboard
— home.html	# Main dashboard analytics & statistics view
— ask.html	# AI Q&A conversational interface
— explain.html	# Custom explanation module
— quiz.html	# Quiz play and scoring interface
— summary.html	# Summarizer tool
— learning_path.html	# Interactive learning roadmaps
— history.html	# Bookmarks and queries history
— profile.html	# User information editor
— settings.html	# API keys and UI preferences
— static/	
— css/	# Modular styling files
— variables.css	# Theme colors & tokens
— base.css	# Global styles & resets
— landing.css	# Welcome page animations
— auth.css	# Login/Signup forms glassmorphism
— dashboard.css	# Dashboard application structure
— animations.css	# Dynamic UI transitions
— dark-mode.css	# Dark mode theme overrides
— js/	# Modular client-side JavaScript controllers
— main.js	# Application initialization
— theme.js	# Dark/light mode switcher
— auth.js	# Handles signup & login API requests
— ask.js	# Manages question submissions & streaming
— explain.js	# Concept explanation logic
— quiz.js	# Interactive quiz controller
— summary.js	# Formats summary generation requests
— learning_path.js	# Renders timelines & schedules
— speech.js	# Speech-to-text / Text-to-speech engine
— toast.js	# User notification popups
— settings.js	# Saves configuration changes
— storage/	# Folder for local data persistence
— data/	# Local database state (.json files)
— uploads/	# Uploaded files for document parsing
— .env.example	# Template for configuration keys
— requirements.txt	# Python application package dependencies
— README.md	# Setup instructions and run configurations
— run.py	# Development server launcher (uvicorn)
— main.py	# FastAPI application entry configuration

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	
Login Credentials (Demo)	username: testuser password: password123
Platform / Hosting Provider	Local deployment (exposable via Ngrok/Uvicorn)
Repository Link	https://github.com/balasurya1266/-EduGenie-AI-learning-assistant
Demo Video Link	https://drive.google.com/file/d/1kJ43XmQ0vI6tNKlCeul-IVWpZunJPphJ/view?usp=drive_link

Step 4: Run Instructions

Pre-requisites:

- Python 3.10 or higher
- Pip (Python Package Installer)
- Google Gemini API Key (obtained from Google AI Studio)
- Operating System: Windows, macOS, or Linux
- A modern web browser (Chrome, Firefox, Edge, Safari)

Step 1: Open your terminal and navigate to the project directory:
`cd EduGenie`

Step 2: Create a Python virtual environment:
`python -m venv venv`

Step 3: Activate the virtual environment:

- Windows (PowerShell): `venv\Scripts\Activate.ps1`
- Windows (CMD): `venv\Scripts\activate.bat`
- macOS / Linux: `source venv/bin/activate`

Step 4: Install all the python dependencies:
`pip install -r requirements.txt`

Step 5: Create your environment configuration file:

- On Windows: copy `.env.example` to `.env`
- On macOS/Linux: cp `.env.example` to `.env`

(Note: Open the `.env` file in a text editor and add your `GEMINI_API_KEY` and a secure `SECRET_KEY` for cookie encryption).

Step 6: Start the local application development server:
python run.py

Step 7: Open your web browser and access the platform at:
<http://localhost:8000>

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Local LaMini loading takes substantial system RAM.	Fallback to Gemini API if local memory constraints arise.